

实验手册一：串程序编写

完成下列两个子任务，形成一个完整的实验报告。

1. 稠密矩阵-稠密矩阵乘 (GEMM)的串行实现

(1) 实验目的

- 理解稠密矩阵-稠密矩阵乘的数学定义
- 熟悉 GEMM 的通用形式
- 掌握 GEMM 的串行实现方法
- 为后续并行化优化实验打下基础

(2) 背景知识

设矩阵 $A \in \mathbb{R}^{m \times k}$ ，矩阵 $B \in \mathbb{R}^{k \times n}$ ，则结果矩阵 $C \in \mathbb{R}^{m \times n}$ 的计算公式如下：

$$C_{ij} = \sum_{p=0}^{k-1} A_{ip} \cdot B_{pj}, \quad (0 \leq i < m, 0 \leq j < n)$$

GEMM 计算示意图如下所示：

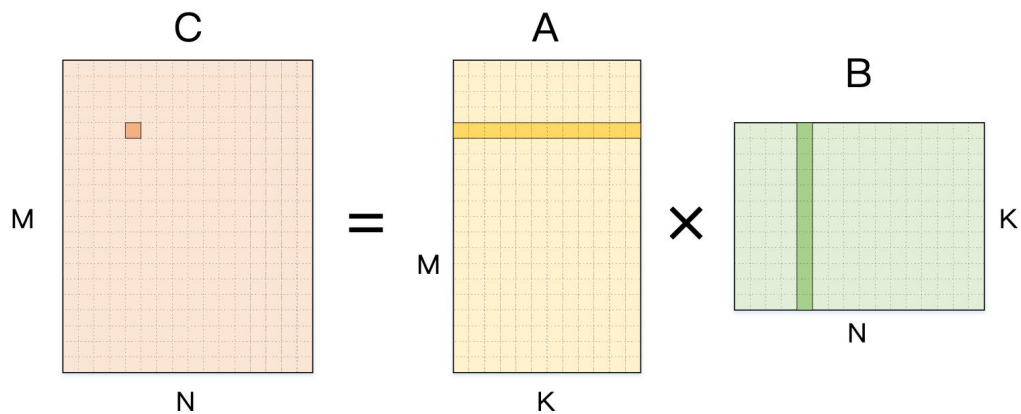


图 1:计算一个结果元素的示意图

在基础线性代数库（BLAS）中，GEMM 被定义为 $C = \alpha \cdot A \cdot B + \beta \cdot C$ ，其中 α, β 为标量系数，这一定义允许在已有结果的基础上更新 C，比单纯的 $C=AB$ 更灵活。

(3) 实验内容

采用 C 语言实现串行的 GEMM 程序（ $C=AB$ ），要求如下：

- 三个矩阵均为方阵，矩阵 A 和 B 的值可以通过随机函数初始化
- 矩阵规模灵活可调，比如通过全局变量、编译参数或运行参数可以快速指定和修改矩阵规模
- 输入矩阵和输出矩阵元素的类型为 `double`，计算精度也为 `double`
- 可通过调用开源线性代数库中的 GEMM 接口计算得到正确的结果矩阵，和自己实现的 GEMM 接口的计算结果进行对比，以证明串行程序实现的正确性。
- 分别设置矩阵规模为 64、128、256、512、1024、2048，记录不同规模下 GEMM 程序的计算时间（不包括矩阵内存申请/释放和矩阵元素初始化的时间）
- 分别尝试 `clock` 函数和 `gettimeofday` 函数两种计时方法
- 计算程序实现的 GFLOPS 和硬件本身的理论 GFLOPS 并进行对比
- 将相关实验数据通过图表形式在实验报告中展示并进行分析。

2. 稀疏矩阵向量乘（SpMV）的串行实现

（1）实验目的

- 理解稀疏矩阵（Sparse Matrix）的基本概念与应用背景。
- 掌握稀疏矩阵常用存储格式 CSR。
- 熟悉稀疏矩阵向量乘（SpMV）的基本流程与实现方法。
- 为后续并行化实验（PThread/OpenMP/MPI/CUDA 等）打下基础。

（2）背景知识

稀疏矩阵是指矩阵中绝大多数元素为 0，仅有少量非零元素。这一类矩阵采用稠密格式存储时会造成内存与计算资源的显著浪费，因此需要稀疏存储格式来高效表示和计算。稀疏矩阵样式具体见图 2 所示。

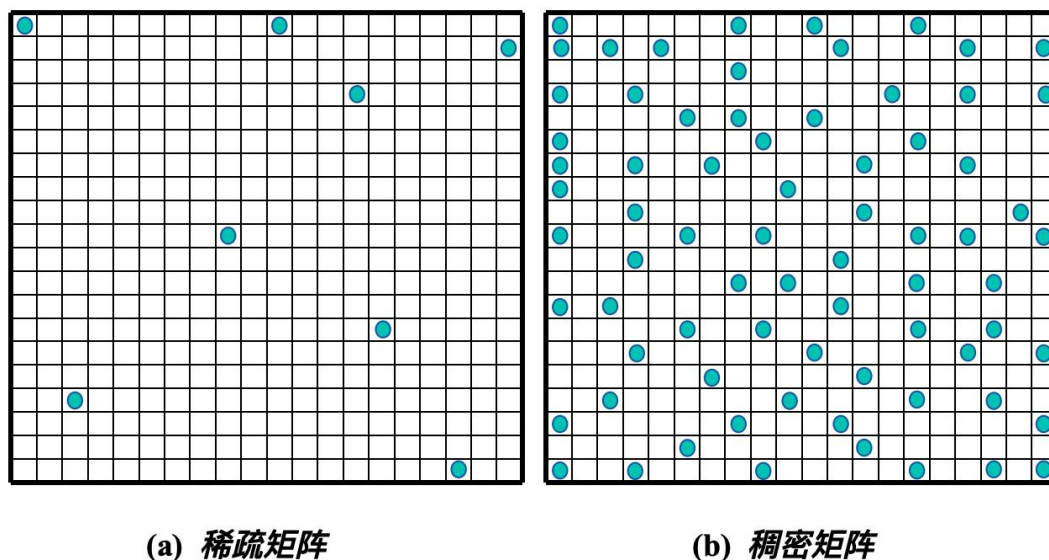


图 2 稀疏矩阵与稠密矩阵非零元素分布对比

稀疏矩阵向量乘（SpMV）可以表示为 $y = A \cdot x$ ，其中 A 为稀疏矩阵， x 为稠密的输入向量， y 为稠密的输出向量。它仅涉及非零元素的乘加运算，其内存访问模式复杂，访存效率往往成为瓶颈，因此属于内存带宽受限性算子，是高性能计算（HPC）领域的重要算子。SpMV 示意图如下所示：

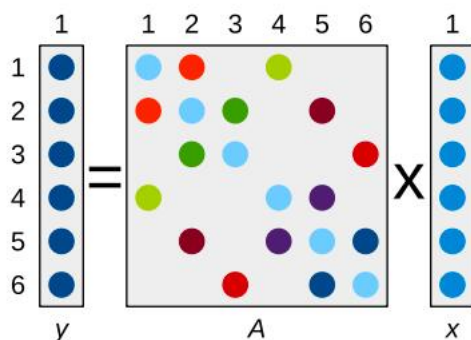
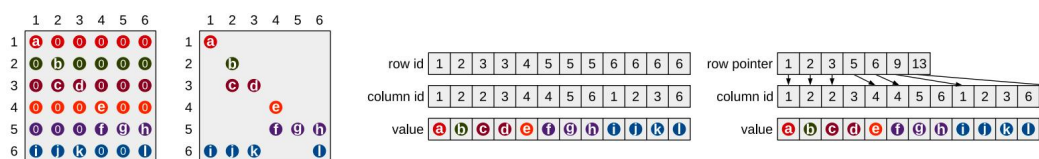


图 3 稀疏矩阵向量乘 (SpMV) 示意图

COO 和 CSR 是常见稀疏矩阵存储格式。

- **COO**: 用三个数组 (row, col, value) 存储每个非零元素的行索引、列索引和值。
- **CSR**: CSR 格式和 COO 格式的列索引数组和值数组相同, 对 COO 格式的行索引数组进行了进一步压缩, 是目前工程与科研中最常见的稀疏矩阵存储格式。

用 CSR 存储一个稀疏矩阵需要三个数组, 首先是 `value[]`, 它用于存储所有非零元素, 按行顺序依次存放, 其次是 `column_id[]`, 它用于存储每个非零元素对应的列索引, 最后是 `row_pointer[]`, 它是行指针数组, 长度为 (行数+1), `row_pointer[i]` 表示第 *i* 行的第一个非零元素在 `value` 数组中的位置, `row_pointer[i+1] - row_pointer[i]` 就是第 *i* 行非零元素的个数。具体例子如下图所示。



(a) 稠密形态 (b) 稀疏形态 (c) 坐标 (COO) 格式 (d) 压缩稀疏行 (CSR) 格式

图 4 稀疏矩阵格式存储示例

(实际存储中 `row_id`、`column_id`、`row_pointer` 的值从 0 开始)

基于 CSR 的 SpMV 算法步骤 (假设 A 和 x 已完成初始化)

- 初始化 `y` 向量全部为 0

- 第一层循环遍历矩阵每一行 i ，第二层循环遍历第 i 行所有非零元的索引 j (j 的取值为 $\text{row_pointer}[i] \sim \text{row_pointer}[i+1]$)，执行

$$y[i] += \text{value}[j] \times x[\text{column_id}[j]]$$

- 遍历完所有行后，得到结果向量 y 。

(3) 实验内容

采用 C 语言实现串行的 SpMV 程序 ($y=Ax$)，要求如下：

- 稀疏矩阵规模（行/列、非零元数）和非零元稠密度（非零元素占比）灵活可调，比如通过全局变量、编译参数或运行参数可以快速修改矩阵规模和非零元稠密度
- 稀疏矩阵 A 为方阵，矩阵 A 和向量 x 的值可以通过随机函数初始化，矩阵 A 的非零元可通过指定的稠密度来生成每行的非零元素数量
- A 、 x 和 y 的元素类型为 `double`，计算精度也为 `double`
- 可通过调用开源线性代数库中的 SpMV 接口计算得到正确的结果矩阵，和自己实现的 SpMV 接口的计算结果进行对比，以证明串行程序实现的正确性。
- 分别设置矩阵规模为 1024、10240、102400、1024000，稀疏矩阵稠密度为 1%、0.1%、0.01%，记录不同规模和稀疏度下 SpMV 程序的计算时间（不包括矩阵/向量内存申请/释放和矩阵/向量元素初始化的时间）
- 分别尝试 `clock` 函数和 `gettimeofday` 两种计时方法
- 计算程序实现的 GFLOPS 和硬件本身的理论 GFLOPS 并进行对比
- 将相关实验数据通过图表形式在实验报告中展示并进行分析。